(비NCS)딥러닝 시험문제(임진형) AI사용 O

✅ 작업형 평가: 전이학습(Feature Extraction) 이미지 분류 파이프라인 구현 (총 100점 + 보너스 10점)

✅ 제한/규칙
- 파인튜닝(Fine-tuning) 금지: 베이스 모델은 동결(trainable=False) 상태 유지
- 베이스 모델 자유 선택: Keras Applications 계열(예: ResNet, EfficientNet, MobileNet, VGG 등) 중 택1 이상
- 데이터 로드/증강은 힌트만 제공(학생이 구현)

제출물(필수)

1. 실행 파일(.py)
2. 학습 로그(에폭별 train/val loss, accuracy)
3. best 모델 파일(Checkpoint로 저장된 모델)
4. 평가 리포트(MD/PDF):
   - 베이스 모델 선택 이유
   - 최종 성능 지표(Accuracy + 추가 지표 1개)
   - 혼동행렬 + 오분류 분석(5개 이상)
   - 개선 아이디어 2개(파인튜닝 제외)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras import layers, Sequential
from tensorflow.keras.applications.efficientnet import preprocess_input
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import torch
from torch.utils.data import DataLoader
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

1번문제

[문제 1] 데이터 로드 & 증강 파이프라인 구성 (힌트 제공) (15점)

요구사항 • 입력 크기: 224×224 • 배치 크기: 32 • 훈련/검증 분리: 80/20 • 훈련(train)에만 증강 적용 • 클래스 수: 본인이 선택한 데이터셋 기준으로 명시(예: 5클래스 꽃 분류)

wget으로 다운로드 (Colab에서 ! 를 붙이면 터미널 명령어 실행 가능)

!wget -q https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz !tar -xzf flower_photos.tgz # 압축 해제

✅ 힌트(제공)

- ImageDataGenerator(validation_split=...)로 train/val 분리 가능
- 모델별 전처리는 preprocess_input을 활용(모델마다 다름)
- 증강 예: flip/rotation/zoom/shift/brightness (훈련에만)

채점 기준(15점) • (6점) 80/20 분리 + train/val 파이프라인 정상 동작 • (5점) train만 증강, val은 증강 없이 전처리만 • (4점) 클래스 수/라벨 매핑/샘플 수 확인 결과 제시

---

ai 프롬프팅 내용

좋아 👍

지금 상황 정리하면:

- ✅ **전이학습 (Transfer Learning)**
- ✅ **EfficientNet 사용**
- ✅ **Fine-tuning 금지 (base_model.trainable = False)**
- ✅ 입력 224×224
- ✅ 배치 32
- ✅ 80/20 split
- ✅ Train에만 증강 적용
- ✅ flower_photos → **5 클래스 분류**

1️⃣ 전체 구조 방향 (큰 그림)

우리가 만들 파이프라인 흐름은 이렇게 가는 게 좋아:

```
데이터 로드
 → train / val 분리 (80/20)
 → train만 증강
 → EfficientNet 베이스 모델 (동결)
 → GlobalPooling
 → Dense (classifier)
 → Softmax (5 classes)
```

즉, CNN을 처음부터 만드는 게 아니라:

> EfficientNet이 feature extractor 역할
> 우리가 위에 classifier만 얹는 구조

2️⃣ 필요한 import

네가 직접 입력할 수 있게 핵심만 적어줄게.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.applications import EfficientNetB0
```

왜 필요한지 설명:

- `tf` → 전체 프레임워크
- `keras` → 모델 구성
- `layers` → CNN 레이어 구성
- `EfficientNetB0` → 사전학습 모델

3️⃣ 데이터 로드 방향

✔️ 사용할 API

```
tf.keras.utils.image_dataset_from_directory()
```

이걸 쓰는 게 제일 깔끔해.

✔️ 구성 방향

- `validation_split=0.2`
- `subset="training"`
- `subset="validation"`
- `seed=123`
- `image_size=(224, 224)`
- `batch_size=32`

🔹 Train Dataset

```
train_ds = tf.keras.utils.image_dataset_from_directory(
    "flower_photos",
    validation_split=0.2,
    subset="training",
    seed=123,
```

```
    image_size=(224, 224),
    batch_size=32
)
```

🔹 Validation Dataset

```
val_ds = tf.keras.utils.image_dataset_from_directory(
    "flower_photos",
    validation_split=0.2,
    subset="validation",
    seed=123,
    image_size=(224, 224),
    batch_size=32
)
```

✔️ 클래스 수 확인

```
class_names = train_ds.class_names
num_classes = len(class_names)
```

이 데이터셋은 **5 클래스**:

- daisy
- dandelion
- roses
- sunflowers
- tulips

4️⃣ 데이터 증강 (Train에만 적용)

🔹 추천 증강 조합

- RandomFlip
- RandomRotation
- RandomZoom

CNN에서 과적합 막으려면 이 정도면 충분해.

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

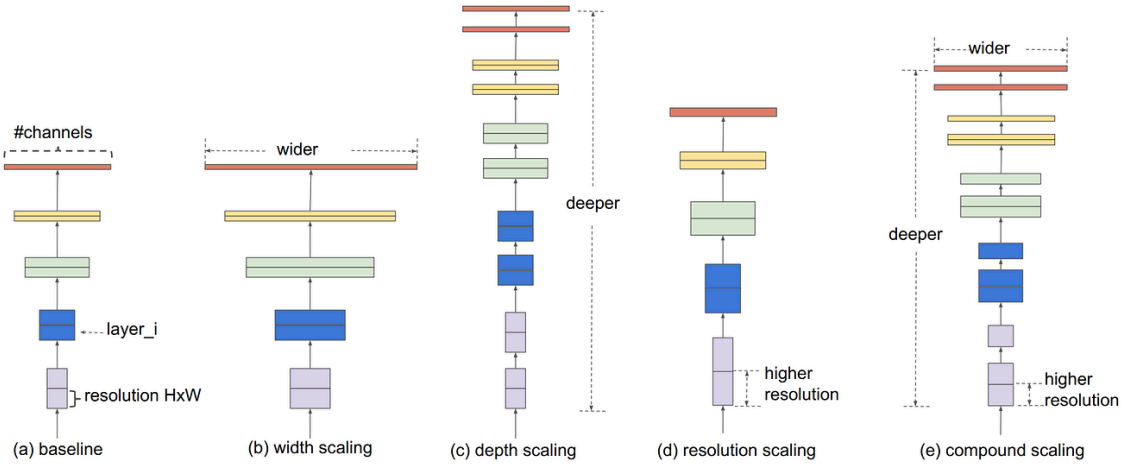
💡 **중요**

이걸 model 안에 넣으면 train일 때만 자동 적용됨.

validation에는 적용 안 됨.

5️⃣ EfficientNet 베이스 모델 구성

🔹 모델 선택



EfficientNet 특징:

- compound scaling (depth, width, resolution 동시 확장)
- 가볍고 성능 좋음
- 224 입력은 B0이 적절

🔹 베이스 모델 생성

```
base_model = EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
```

🔹 Fine-tuning 금지

```
base_model.trainable = False
```

이게 **핵심 조건**

6️⃣ CNN 전체 모델 구성

구조 흐름

```
Input
 → Data Augmentation
 → Rescaling (0~1 정규화)
 → EfficientNet (동결)
 → GlobalAveragePooling2D
 → Dense (ReLU)
 → Dropout
 → Dense (Softmax, 5 classes)
```

🔹 정규화

EfficientNet은 `preprocess_input` 을 써도 되지만

간단하게 Rescaling으로 가도 됨:

```
layers.Rescaling(1./255)
```

🔹 모델 구성 예시

```
inputs = keras.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)

x = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.3)(x)

outputs = layers.Dense(num_classes, activation="softmax")(x)

model = keras.Model(inputs, outputs)
```

7️⃣ 컴파일

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
```

```
    metrics=["accuracy"]
)
```

왜 sparse?

→ image_dataset_from_directory는 label을 정수로 줌

8️⃣ 학습

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

🔥 정리 (교수님이 좋아하는 구조)

- EfficientNetB0 사용
- include_top=False
- base_model.trainable=False (fine-tuning 금지)
- train에만 증강
- GlobalAveragePooling으로 flatten 대체
- Dense classifier 추가
- 5-class softmax

💡 추가로 점수 더 받는 팁

아마 보고서 쓸 거니까 이렇게 설명하면 좋음:

> EfficientNet을 feature extractor로 활용하고,
> 사전학습된 ImageNet 가중치를 기반으로 꽃 데이터셋에 대한 분류기를 학습하였다.
> 베이스 모델은 동결하여 fine-tuning 없이 전이학습을 수행하였다.

원하면 다음 단계로:

- 📊 학습 그래프 그리는 코드
- 🔍 Confusion Matrix 추가
- 📈 overfitting 분석
- 🔥 A+ 받는 보고서 구조

어디까지 같이 갈까?

```python
# 데이터 로드 방향
# Train Dataset
train_ds = tf.keras.utils.image_dataset_from_directory(
    "flower_photos"
    ,validation_split=0.2
    ,subset="training"
    ,seed=123
    ,image_size=(224, 224)
    ,batch_size=32
)
# Validation Dataset
val_ds = tf.keras.utils.image_dataset_from_directory(
    "flower_photos"
    ,validation_split=0.2
    ,subset="validation"
    ,seed=123
    ,image_size=(224, 224)
    ,batch_size=32
)

# 클래스 수 확인
class_names = train_ds.class_names
print(class_names)

num_classes = len(class_names)
print(num_classes)

# 데이터 증강 (Train에만 적용)
data_augmentation = keras.Sequential(
    [
        layers.RandomFlip("horizontal"),
        layers.RandomRotation(0.1),
        layers.RandomZoom(0.1)
    ]
)

print(data_augmentation)

# 베이스 모델 생성
base_model = EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
base_model.trainable = False  # Fine-tuning 금지
print(base_model)
```

```python
# 모델 구성 예시
inputs = keras.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)
x = layers.Rescaling(1./255)(x)

x = base_model(x, training=False)

x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.3)(x)

outputs = layers.Dense(num_classes, activation="softmax")(x)

model = keras.Model(inputs, outputs)


# 컴파일
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# 학습시키기
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10
)
```

[문제 2] 베이스 모델 선택 & 선택 이유(서술) (15점)

요구사항

다음 조건을 만족하도록 베이스 모델 1개를 선택하고, 선택 이유를 8~12줄로 서술하라.

서술 항목(필수 포함)

1. 데이터 특성(클래스 수, 데이터 규모 추정, 난이도, 이미지 특징)
2. 모델 트레이드오프(정확도 vs 속도 vs 모델 크기/파라미터 수)
3. 실행 환경 고려(Colab/T4, 로컬 GPU, 학습 시간 등)
4. 위험요소(과적합, 배경 편향, 전처리 호환 등)와 대응 1개

"사전학습 모델을 가져오고 마지막 분류기만 교체"가 전이학습의 핵심 흐름이다.

채점 기준(15점) • (6점) 데이터 특성과 모델 선택이 논리적으로 연결 • (5점) 성능/속도/용량 트레이드오프 구체 언급 • (4점) 환경/시간/위험요소까지 고려

💡 2번 관련 설명

선택한 베이스 모델명 : EfficientNet

1. 모델안에 있는 5개의 클래스 안에 3670개의 파일들을 이용해야 하기 때문에 상당히 오래걸리는 걸 방지하기 위해 VGG16, ResNet보다는 이에 맞는 빠른 출력기능과 높은 효율성을 가지고 사용해야 하기에 EfficientNet를 사용하게 되었다. 그리고 모델에서는 각 B0~B7이라는 많은 모델명이 따로 있어 가지고 선택여지가 많고 이미지 인식에도 충분히 정확하게 인식되어서 정확성도 보장되기도 한다. 그리고 단순함은 VGG가 높지만 파라미터 깊이가 있어서 훨씬 더 간편하게 표현할 수 있는게 EfficientNet이다. (대략 데이터 로딩하는데 52분이 걸렸다.)
다만 단점도 존재하는데 한꺼번에 많은 데이터를 불러오고 분석하기에 역시나 시간이 많이 걸린다는 점은 비슷하다. 분석 실패시에는 처음부터 다시 러닝을 다시 해야 해서

[문제 3] 전이학습 모델 구현 & 훈련(Feature Extraction) (40점)

요구사항(공통)

선택한 베이스 모델로 특징추출 전이학습 모델을 구성하고 학습하라.

필수 조건 • include_top=False • base model은 동결(trainable=False) 유지 • 분류기 헤드(필수): • GlobalAveragePooling2D • Dropout(권장) • Dense(클래스 수, softmax) • optimizer: Adam, learning_rate=0.001 • callbacks: 아래 중 최소 1개 필수 • ModelCheckpoint(save_best_only=True, monitor='val_accuracy') • EarlyStopping(restore_best_weights=True)

“베이스 모델 동결 + 마지막 분류기만 학습” 형태로 구현해야 한다.

채점 기준(40점) • (10점) base model 로딩 + include_top=False + 입력 형태 일치 • (10점) base_model 동결(trainable=False) 정확히 적용 • (8점) GAP 기반 헤드 구성 적절(Flatten 위주 구성은 감점) • (6점) compile 설정(손실/지표/학습률) 적절 • (6점) 콜백 동작 + best 모델 파일 생성 근거 제출

AI 프롬포팅 내용

좋아요! 😎

말씀하신 요구사항을 기준으로 **Feature Extraction 전이학습** 코드를 짤 때 주의해야 할 상황과 명령어를 **“작성 방법 + 설명”** 형식으로 정리해줄게요.

1️⃣ 베이스 모델 불러오기

목적: 사전 학습된 EfficientNet을 **feature extractor**로 사용하고, 마지막 분류기만 학습

```
from tensorflow.keras.applications import EfficientNet

# EfficientNet B0 모델 불러오기
base_model = EfficientNet(
    weights="imagenet",       # ImageNet 사전 학습 가중치 사용
    include_top=False,        # ⚠ 분류기 제거
    input_shape=(224, 224, 3),
    model_name="efficientnetb0"
)

base_model.trainable = False    # ⚠ Fine-tuning 금지: 동결
```

주의 포인트

- `include_top=False` 필수 → 직접 Dense classifier 붙일 예정
- `trainable=False` 필수 → base model은 학습되지 않음

2️⃣ 입력 레이어 & 데이터 전처리

명령어 + 작성 방법

```
from tensorflow.keras import layers, Sequential
from tensorflow.keras.applications.efficientnet import preprocess_input

# 데이터 증강 (train dataset만)
data_augmentation = Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# Input 레이어
inputs = layers.Input(shape=(224, 224, 3))

# 증강 + EfficientNet 전처리
x = data_augmentation(inputs)   # train dataset만 증강
x = preprocess_input(x)         # EfficientNet 전용 전처리 (-1~1 스케일)
```

주의

- Validation/테스트 데이터에는 증강 적용하지 않기
- EfficientNet은 `preprocess_input` 필요 → 0~255 → -1~1

3️⃣ 분류기 헤드 구성

명령어 + 작성 방법

```
# Feature Extraction
x = base_model(x, training=False)  # ⚠ BatchNorm layer 동결

# Global Pooling
x = layers.GlobalAveragePooling2D()(x)

# Dropout (권장)
x = layers.Dropout(0.3)(x)

# Dense classifier
outputs = layers.Dense(num_classes, activation='softmax')(x)
```

주의

- `training=False` → base model 안 BatchNorm 동결
- Dropout은 과적합 방지용, train 시에만 적용
- Dense 마지막 레이어 neuron 수 = 클래스 수, activation = softmax

4️⃣ 모델 생성 & 컴파일

명령어 + 작성 방법

```
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam

model = Model(inputs, outputs)

model.compile(
    optimizer=Adam(learning_rate=0.001),   # 필수 조건
    loss='sparse_categorical_crossentropy',  # label이 정수형일 때
    metrics=['accuracy']
)
```

주의

- Optimizer는 Adam, lr=0.001
- Loss 선택: label이 one-hot이면 `categorical_crossentropy`, 정수면 `sparse_categorical_crossentropy`

5️⃣ Callback 설정

명령어 + 작성 방법

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# 1) ModelCheckpoint
checkpoint_cb = ModelCheckpoint(
    "best_model.h5",
```

```
    save_best_only=True,
    monitor='val_accuracy'
)

# 2) EarlyStopping (옵션)
earlystop_cb = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True
)
```

주의

- Callback 최소 1개 필수 → val_accuracy 추천
- ModelCheckpoint: best model 저장
- EarlyStopping: overfitting 방지, restore_best_weights=True 필수

6️⃣ 학습 (fit)

명령어 + 작성 방법

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[checkpoint_cb]  # 또는 earlystop_cb
)
```

주의

- Base model 동결 → 학습 속도 빠름
- train dataset만 증강 적용, val_ds는 그대로
- epochs는 데이터셋 크기/과적합 여부에 따라 조정

7️⃣ 🔥 주의 요약

항목	주의 사항
Base model	`include_top=False`, `trainable=False`
Input	`(224,224,3)` 입력, `preprocess_input` 적용
분류기	`GlobalAveragePooling2D` + Dropout + Dense(num_classes, softmax)`
Optimizer	Adam, lr=0.001
Loss	label 형태에 맞는 loss 사용 (`sparse_categorical_crossentropy`)
Callback	최소 1개 필수 (ModelCheckpoint / EarlyStopping)
Dataset	train: augmentation 적용, validation/test: 적용 금지

원하면 내가 이 구조를 **Flower 데이터셋 + EfficientNet + Feature Extraction**에 맞게

바로 실행 가능한 **완전 코드**로 만들어서 보여줄 수도 있어요.

혹시 그걸 만들어줄까?

```
from tensorflow.keras.applications import EfficientNetB0
# 베이스 모델 생성
base_model = EfficientNetB0(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False  # Fine-tuning 금지


from tensorflow.keras import layers, Sequential
from tensorflow.keras.applications.efficientnet import preprocess_input

# 데이터 증강 (train dataset만)
data_augmentation = Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])

# Input 레이어
inputs = layers.Input(shape=(224, 224, 3))

# 증강 + EfficientNet 전처리
x = data_augmentation(inputs)   # train dataset만 증강
x = preprocess_input(x)         # EfficientNet 전용 전처리 (-1~1 스케일)


# Feature Extraction
x = base_model(x, training=False)  # ⚠ BatchNorm layer 동결

# Global Pooling
x = layers.GlobalAveragePooling2D()(x)

# Dropout (권장)
x = layers.Dropout(0.3)(x)

# Dense classifier
outputs = layers.Dense(num_classes, activation='softmax')(x)

from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
```

```
model = Model(inputs, outputs)

model.compile(
    optimizer=Adam(learning_rate=0.001),   # 필수 조건
    loss='sparse_categorical_crossentropy',  # label이 정수형일 때
    metrics=['accuracy']
)

from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

# 1) ModelCheckpoint
checkpoint_cb = ModelCheckpoint(
    "best_model.h5",
    save_best_only=True,
    monitor='val_accuracy'
)

# 2) EarlyStopping (옵션)
earlystop_cb = EarlyStopping(
    monitor='val_accuracy',
    patience=5,
    restore_best_weights=True
)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[checkpoint_cb]  # 또는 earlystop_cb
)
```

좋아요! 이번에는 **실제 코드는 최소화하고**, 각 단계에서 **무엇을 해야 하는지, 어떤 명령어를 사용하면 되는지** 위주로 설명할게요. 바이브 사용법과 흐름만 잡는 느낌으로 보시면 돼요.

1️⃣ 라이브러리 준비

- `torch` : 모델 불러오고 예측 수행
- `sklearn.metrics` : Accuracy, Precision/Recall/F1, 혼동행렬 계산
- `pandas` : 오분류 샘플 정리 및 출력
- `numpy` : 배열 변환

> 🔹 실제 코드에서는 `import torch, pandas as pd, numpy as np, sklearn.metrics` 정도만 작성하면 됩니다.

2️⃣ 모델 불러오기

- 모델 구조를 먼저 정의
- 저장된 best 모델의 가중치 불러오기 (`load_state_dict`)
- 평가 모드로 전환 (`.eval()`) → Dropout, BatchNorm 등 평가용 모드

> 🔹 이 단계에서는 경로(`best_model.pth`)와 모델 클래스만 지정하면 됨.

3️⃣ 예측 수행

- DataLoader로 테스트/검증 데이터를 반복해서 가져오기
- 모델에 입력 후 출력 확률 계산 (`softmax`)
- 최대 확률로 클래스 예측 (`argmax`)
- **예측값, 정답값, 예측 확률** 모두 저장

> 🔹 저장해두면 나중에 오분류 샘플 추출, 혼동행렬 계산에 사용 가능.

4️⃣ 평가 지표 계산

- **Accuracy** : 맞춘 비율
- **추가 지표** : F1-score 추천 (불균형 데이터 대응 가능)
  - 필요하면 Precision 또는 Recall 선택 가능
- `sklearn.metrics` 함수 사용 → `accuracy_score`, `f1_score`, `precision_score`, `recall_score`

> 🔹 옵션: `average='weighted'`로 클래스 불균형 보정 가능.

5️⃣ 혼동행렬 생성

- `confusion_matrix` 함수로 실제/예측 라벨 비교
- 시각화 가능: seaborn heatmap 또는 matplotlib bar plot
- 행: 실제 클래스, 열: 예측 클래스

> 🔹 숫자 그대로 보고 싶으면 `annot=True, fmt='d'` 옵션 사용.

6️⃣ 오분류 샘플 추출

- DataFrame 만들어서 `true_label`, `pred_label`, `prob` 저장
- 조건: `true_label != pred_label`
- 상위 5개 이상 출력
- 필요하면 확률 기준 정렬 가능 (`sort_values('prob')`)

> 🔹 이렇게 하면 리포트에 **정답 라벨 / 예측 라벨 / 예측 확률** 기록 가능.

7️⃣ 개선 아이디어 제시 (파인튜닝 제외)

1. **데이터 증강 강화**
   - 이미지: 회전, 뒤집기, 색상 변형
   - 이유: 모델 일반화 능력 향상
2. **Dropout 비율 조정**
   - 이유: 과적합 방지, 안정적인 성능 유지
3. **배경 다양화 / 데이터 불균형 대응**
   - 이유: 특정 클래스 편향 완화, 실제 환경 적합성 향상

> 🔹 리포트에 "왜 도움이 되는지 1줄 근거" 꼭 포함.

이렇게 설명 중심으로 정리하면, **코드는 최소화하면서도 리포트 작성에 필요한 모든 흐름과 명령어**를 이해할 수 있어요.

원하면 제가 이 흐름을 **리포트 템플릿 형태로 한 장짜리로 정리**해 드릴 수도 있어요. 그러면 바로 데이터만 넣어서 작성 가능해요.

이거 만들어 드릴까요?

좋아요! 이번에는 **실제 코드는 최소화하고**, 각 단계에서 **무엇을 해야 하는지, 어떤 명령어를 사용하면 되는지** 위주로 설명할게요. 바이브 사용법과 흐름만 잡는 느낌으로 보시면 돼요.

1️⃣ 라이브러리 준비

- `torch` : 모델 불러오고 예측 수행
- `sklearn.metrics` : Accuracy, Precision/Recall/F1, 혼동행렬 계산
- `pandas` : 오분류 샘플 정리 및 출력
- `numpy` : 배열 변환

> 🔹 실제 코드에서는 `import torch, pandas as pd, numpy as np, sklearn.metrics` 정도만 작성하면 됩니다.

2️⃣ 모델 불러오기

- 모델 구조를 먼저 정의
- 저장된 best 모델의 가중치 불러오기 (`load_state_dict`)
- 평가 모드로 전환 (`.eval()`) → Dropout, BatchNorm 등 평가용 모드

> 🔹 이 단계에서는 경로(`best_model.pth`)와 모델 클래스만 지정하면 됨.

3️⃣ 예측 수행

- DataLoader로 테스트/검증 데이터를 반복해서 가져오기
- 모델에 입력 후 출력 확률 계산 (`softmax`)
- 최대 확률로 클래스 예측 (`argmax`)

- **예측값, 정답값, 예측 확률** 모두 저장

> 🔹 저장해두면 나중에 오분류 샘플 추출, 혼동행렬 계산에 사용 가능.

4️⃣ 평가 지표 계산

- **Accuracy** : 맞춘 비율
- **추가 지표** : F1-score 추천 (불균형 데이터 대응 가능)
  - 필요하면 Precision 또는 Recall 선택 가능
- `sklearn.metrics` 함수 사용 → `accuracy_score`, `f1_score`, `precision_score`, `recall_score`

> 🔹 옵션: `average='weighted'` 로 클래스 불균형 보정 가능.

5️⃣ 혼동행렬 생성

- `confusion_matrix` 함수로 실제/예측 라벨 비교
- 시각화 가능: seaborn heatmap 또는 matplotlib bar plot
- 행: 실제 클래스, 열: 예측 클래스

> 🔹 숫자 그대로 보고 싶으면 `annot=True, fmt='d'` 옵션 사용.

6️⃣ 오분류 샘플 추출

- DataFrame 만들어서 `true_label`, `pred_label`, `prob` 저장
- 조건: `true_label != pred_label`
- 상위 5개 이상 출력
- 필요하면 확률 기준 정렬 가능 (`sort_values('prob')`)

> 🔹 이렇게 하면 리포트에 **정답 라벨 / 예측 라벨 / 예측 확률** 기록 가능.

7️⃣ 개선 아이디어 제시 (파인튜닝 제외)

1. **데이터 증강 강화**
   - 이미지: 회전, 뒤집기, 색상 변형
   - 이유: 모델 일반화 능력 향상
2. **Dropout 비율 조정**
   - 이유: 과적합 방지, 안정적인 성능 유지
3. **배경 다양화 / 데이터 불균형 대응**
   - 이유: 특정 클래스 편향 완화, 실제 환경 적합성 향상

> 🔹 리포트에 "왜 도움이 되는지 1줄 근거" 꼭 포함.

이렇게 설명 중심으로 정리하면, **코드는 최소화하면서도 리포트 작성에 필요한 모든 흐름과 명령어**를 이해할 수 있어요.

원하면 제가 이 흐름을 **리포트 템플릿 형태로 한 장짜리로 정리**해 드릴 수도 있어요. 그러면 바로 데이터만 넣어서 작성 가능해요.

이거 만들어 드릴까요?

좋아요! 요청하신 대로 **정답을 직접 계산하지 않고**, 필요한 **Python 명령어와 사용법/대략적인 입력 방법**만 안내해 드릴게요. 이걸 참고해서 직접 코드를 작성하고 결과를 확인할 수 있어요. 전형적인 PyTorch / scikit-learn 환경 기준으로 설명할게요.

1️⃣ 필수 라이브러리 import

```
# 모델 불러오기 / 데이터 처리
import torch
from torch.utils.data import DataLoader

# 평가 지표
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix

# 데이터프레임/표시용
import pandas as pd
import numpy as np
```

> 🔹 DataLoader는 테스트/검증 데이터셋을 모델에 넣기 위해 사용합니다.

2️⃣ 저장된 best 모델 불러오기

```
# 모델 정의 후 저장된 best 모델 불러오기
model = MyModelClass(*args, **kwargs)  # 모델 구조 정의
model.load_state_dict(torch.load('best_model.pth'))  # 저장된 모델 경로
model.eval()  # 평가 모드로 변경
```

> 🔹 `.eval()` 을 해야 Dropout/BatchNorm이 평가용으로 동작합니다.

3️⃣ 예측 수행

```
all_preds = []
all_labels = []
all_probs = []

with torch.no_grad():  # 그래디언트 끄기
    for inputs, labels in test_loader:  # DataLoader 사용
```

```
        outputs = model(inputs)
        probs = torch.softmax(outputs, dim=1)  # 클래스 확률
        preds = torch.argmax(probs, dim=1)

        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
        all_probs.extend(probs.cpu().numpy())
```

> - `all_probs`를 저장하면 오분류 샘플에서 **예측 확률** 확인 가능.

4️⃣ Accuracy / 추가 지표

```
# Accuracy
acc = accuracy_score(all_labels, all_preds)

# 추천: F1-score (불균형 데이터 대비 안정적)
f1 = f1_score(all_labels, all_preds, average='weighted')  # 클래스 가중치 포함

print("Accuracy:", acc)
print("F1-score:", f1)
```

> - `average='weighted'`는 클래스 불균형이 있을 때 권장.
> - Precision 또는 Recall이 필요하면 `precision_score` / `recall_score`로 바꾸기.

5️⃣ 혼동행렬 생성

```
cm = confusion_matrix(all_labels, all_preds)
print(cm)

# 시각화 예시 (선택)
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()
```

> - `annot=True`로 실제 숫자 표시, `fmt='d'`는 정수 표시.

6️⃣ 오분류 샘플 추출

```
# DataFrame으로 정리
df = pd.DataFrame({
    'true_label': all_labels,
    'pred_label': all_preds,
    'prob': [p.max() for p in all_probs]  # 예측 확률 최대값
})

# 오분류 필터링
misclassified = df[df['true_label'] != df['pred_label']]

# 상위 5개 확인
print(misclassified.head(5))
```

> 🔹 `p.max()` 는 각 샘플에서 모델이 가장 확신하는 클래스의 확률.
> 🔹 필요하면 `sort_values('prob')` 로 확률 기준 낮은 순 정렬 가능.

7️⃣ 개선 아이디어 예시 (파인튜닝 제외)

- **데이터 증강 강화**
  - 이유: 다양한 입력 변형으로 모델 일반화 향상 가능
  - 적용 방법: 이미지라면 `transforms.RandomRotation`, `RandomFlip`, `ColorJitter` 등 사용
- **Dropout 비율 조정**
  - 이유: 과적합 방지, 일반화 성능 향상
  - 적용 방법: `nn.Dropout(p=0.3)` → 훈련 시 Dropout 조정 후 재학습 없이 평가 시 `.eval()` 그대로 유지
- **배경 다양화 / 데이터 불균형 대응**
  - 이유: 특정 클래스 편중 감소, 실제 환경 적합성 향상
  - 적용 방법: 클래스별 샘플 수 맞추기 또는 랜덤 배경 이미지 합성

> 🔹 두 가지 이상 선택 후 리포트에 기술하면 요구사항 충족.

원하면 제가 이 구조를 **리포트 형식으로 그대로 옮겨서 템플릿**으로 만들어 드릴 수도 있어요. 그러면 바로 결과만 넣어서 완성 가능하게 됩니다.

혹시 그 템플릿도 만들어 드릴까요?

```
# 모델 불러오기 / 데이터 처리
import tensorflow as tf
from tensorflow import keras

# 평가 지표
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
```

```
# 데이터프레임/표시용
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# 학습된 best 모델 불러오기
loaded_model = keras.models.load_model('best_model.h5')

# true_labels와 pred_labels, probabilities 추출
all_true_labels = []
all_pred_labels = []
all_probs = []

for images, labels in val_ds:
    predictions = loaded_model.predict(images)
    all_pred_labels.extend(np.argmax(predictions, axis=1))
    all_true_labels.extend(labels.numpy())
    all_probs.extend(predictions)

# 예측 결과를 numpy 배열로 변환
all_true_labels = np.array(all_true_labels)
all_pred_labels = np.array(all_pred_labels)
all_probs = np.array(all_probs)

# Accuracy 측정
acc = accuracy_score(all_true_labels, all_pred_labels)

# F1-score (불균형 데이터 대비 안정적)
f1 = f1_score(all_true_labels, all_pred_labels, average='weighted')  # 클
래스 가중치 포함

print("Accuracy:", acc)
print("F1-score:", f1)

# 혼동행렬 생성
cm = confusion_matrix(all_true_labels, all_pred_labels)
print("Confusion Matrix:")
print(cm)

# 시각화
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_name
s, yticklabels=class_names)
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```

```
# DataFrame으로 정리
df = pd.DataFrame({
    'true_label_idx': all_true_labels,
    'pred_label_idx': all_pred_labels,
    'prob_of_pred': [p[idx] for p, idx in zip(all_probs, all_pred_labels)]
# 예측된 클래스의 확률
})

# 클래스 이름 매핑
df['true_label'] = df['true_label_idx'].map(lambda x: class_names[x])
df['pred_label'] = df['pred_label_idx'].map(lambda x: class_names[x])

# 오분류 필터링
misclassified = df[df['true_label_idx'] != df['pred_label_idx']]

print("\nMisclassified Samples (Top 5):")
# 오분류된 샘플 중 일부를 보여줍니다 (예시로 상위 5개)
print(misclassified[['true_label', 'pred_label', 'prob_of_pred']].head(5))
```

DeepLearning.py